


# BackyardPay

A Mobile Application for Backyard Rental Management and Payment Recordkeeping

iYunivesithi Walter Sisulu

Department of Business and Application Development

Application Technology IV (APT 41P2)

Student: Mbongiseni Nkambule

Student number: 231064977

Project submission: 12 November 2026

**Project links**

GitHub: https://github.com/nkambuleinnocent52-Ghost/Mbongiseni-Nkambule-BackyardPay

Deployed application: https://quiet-pastelito-19fa50.netlify.app

# 1. Context and background

## The rental-record problem

Small backyard rentals in East London, Eastern Cape may use cash, bank transfers, notebooks and messages to record rent. These separate records make it difficult to identify the tenant and room, the month covered, whether money was actually received and the balance still due. A tenant may have bank proof but no shared record showing how the landlord applied the payment. A payment reported by a tenant must therefore be distinguished from a payment confirmed by the landlord.

## Target users

BackyardPay serves landlords managing a small set of properties and rooms, and the tenants renting those rooms. The landlord assigns an available room and supplies a private invitation. The tenant registers against that assigned rental. Each role can access its own rental records.

## Why mobile access helps

A phone lets a tenant report money already paid, check monthly balances, retrieve receipts and describe a repair problem where it occurs. A landlord can check a payment or respond to a problem while away from a desk. The Flutter interface adapts to phone and desktop widths. An internet connection is needed to access shared account records (Flutter, n.d.).

## Scope

The responsive Flutter/Dart application is hosted on Netlify. Supabase supplies authentication and PostgreSQL storage. BackyardPay records payments made by cash, EFT or other supported methods; it does not transfer money or verify bank transactions automatically. Bank proof shows a banking transaction, while the application receipt records the landlord's acknowledgement and the rental allocation. Security deposits are accounted for separately from rent.

Monthly rent is due on the 1st. Where selected, first-month rent is prorated using actual calendar days including move-in day. The application sets a ten-calendar-day deposit deadline. Rental agreements and applicable South African rental law govern the parties' obligations (Republic of South Africa, 1999).

# 2. Goal and objectives

Goal: provide a clear, shared and traceable record of backyard rentals, rent, deposits and reported problems.

1. Register users and securely connect tenants to rooms already assigned by the landlord.

2. Record property details, room availability, room rates, deposits and actual move-in dates.

3. Calculate first-month charges, confirmed monthly balances and proposed allocations of advance rent.

4. Separate tenant reports from landlord-confirmed payments and produce retrievable receipts.

5. Support issue reports, responses, status tracking and notifications linked to the relevant record.

### Input → processing → output

Inputs: property address, room and rental terms; tenant contact and move-in details; payment amount, date, method, reference and purpose; issue location and description.

Processing: check availability and permissions; calculate rent in cents; propose allocations across months; confirm money received; maintain separate rent/deposit records.

Outputs: role dashboards, monthly rental accounts, pending/confirmed/rejected payment histories, receipts, room availability, issue status and notifications.

# 3.1 Landlord: get started and add a property

Start here if you rent rooms to other people. The landlord sets up the property and room before inviting a tenant.

**1.** Open https://quiet-pastelito-19fa50.netlify.app in your internet browser. Select Get started, then Landlord.

**2.** Select Create an account. Enter the details requested and choose a password. If an email confirmation is requested, follow the message sent to your email address, then log in.

**3.** The dashboard is your home screen. Overview returns you here. The menu also has Payments, Tenants, Properties, Notifications and Account.

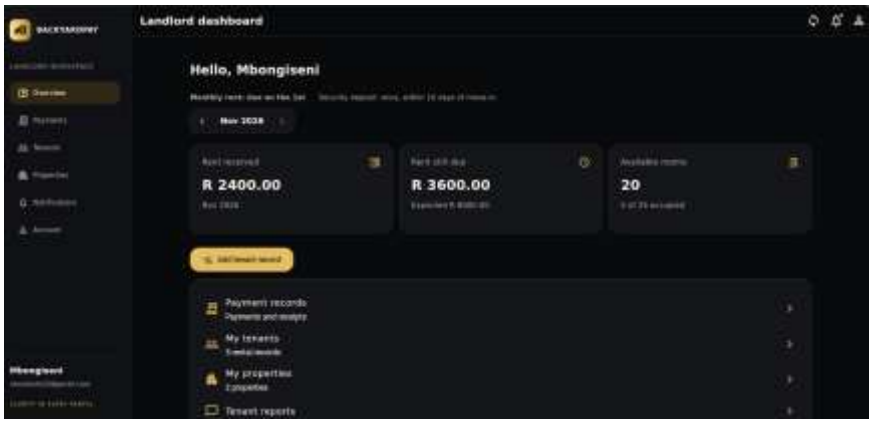



Figure 1. Landlord home screen and main menu.

**4.** Select Properties, then Add property. Enter the property name, full address, number of rooms, monthly rent and security deposit. The deposit is separate from rent. Select Save.

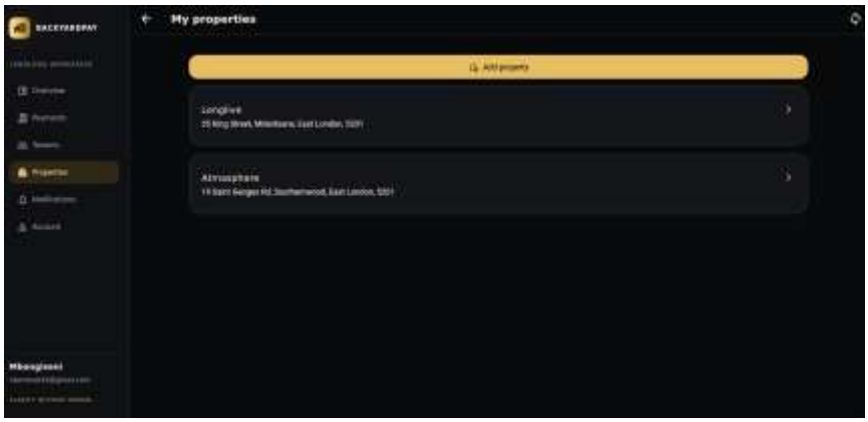



Figure 2. Properties saved by the landlord.

**5.** Select the property name to see its rooms. Use Room rent if a room has a different price. Check the details before adding a tenant.

## 3.2 Landlord: add a tenant and check payments

**1.** Select Tenants, then Add tenant record. Choose the property and an available room. Enter the tenant's name, contact details and actual move-in date.




Figure 3. Form for assigning a tenant to an available room.

**2.** Check the first rent and separate deposit amounts, then save. Open Invite this tenant in the saved rental record. Copy the private invitation and send it only to that tenant. The tenant then follows section 3.3.

**3.** After the tenant reports a payment, open Payments and select the record marked Pending. Pending means it is waiting for you to check.

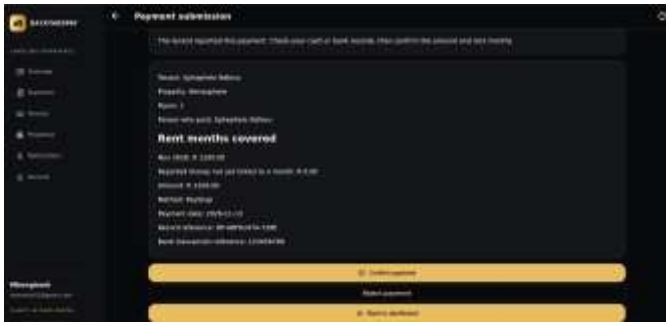



Figure 4. Payment record waiting for the landlord's decision.

**4.** Check the tenant, room, amount and whether the money is for rent or deposit. Check your bank account or cash received. If correct, select Confirm received. If incorrect, select Reject and explain why.




Figure 5. Notifications about payments and rental records.

**5.** Select Notifications and open the relevant notice to see the payment or reported problem. For a problem, acknowledge it, reply and change its status as work progresses: Acknowledged, In progress, then Resolved. Mark as read only clears the unread notice.

## 3.3 Tenant: create your account and find your room

Start here if you are renting a room. Ask your landlord for your private invitation before creating an account.

**1.** Open BackyardPay. Select Get started, then Tenant, then Create an account. Use the invitation your landlord sent and enter the details requested. Follow any email confirmation instructions, then log in.

**2.** Your home screen shows your property, room and the selected month. Check that these details are correct. Tell the landlord if the room or property is wrong.

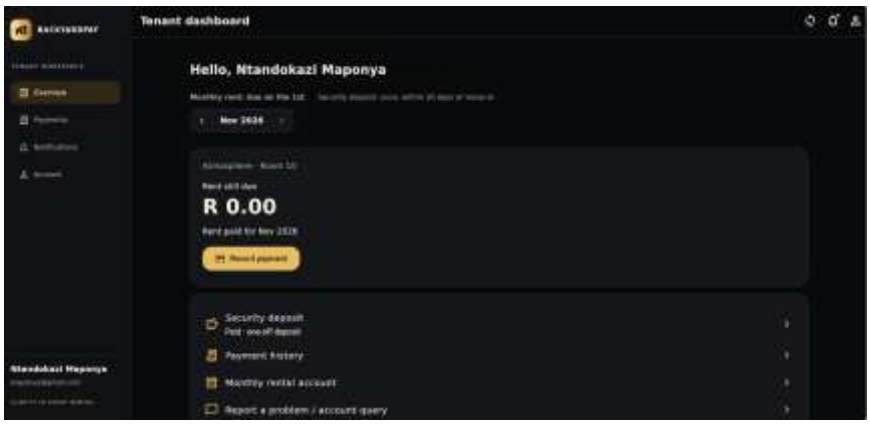



Figure 6. Tenant home screen with rent information and rental links.

**3.** Rent still due is the amount left to pay for the selected month. Use the month arrows to check another month. Select Rental details to see your room and rental information.




Figure 7. Account screen for your personal details and password.

**4.** Select Account to check or edit your contact details or change your password. To leave the account, use the top-right logout icon and select Logout. Select Cancel to stay signed in.

On a phone, open the navigation menu to find these pages. Your browser normally keeps you signed in until you log out or clear its saved data.

## 3.4 Tenant: report money you have paid

**Pay the landlord using your usual cash or banking method first. BackyardPay keeps the record; it does not send money.**

**1.** On your home screen, select Record payment. Choose Rent for monthly rent or Security deposit for deposit money.




Figure 8. Payment form: payment purpose, amount, method and date.

**2.** Enter the amount you paid. Select how you paid and the date you actually paid. For a bank payment, enter the reference shown by your bank. Keep your bank proof of payment.

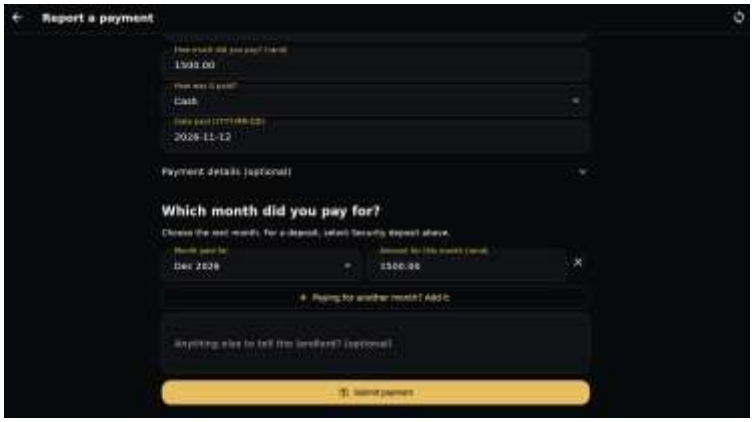



Figure 9. Rent payment split between the selected months.

**3.** For rent, check the month and amount shown. If one payment covers more than one month, check each month's share. The total must match the money you paid.

Example: if you owe R1,200 now and pay R2,000, R1,200 covers this month and R800 goes towards next month. If next month's rent is R1,500, R700 remains after the landlord confirms the payment.

**4.** Select Submit payment once. The record is Pending until the landlord confirms receipt. Open Payments to check the result. Confirmed means the landlord accepted the payment; Rejected means you should read the reason and contact the landlord.

## 3.5 Tenant: check balances, deposit and problems

**1.** Open Monthly rental account from your home screen. Choose the month. Check the rent charged, confirmed payments and rent still due. Monthly rent is due on the 1st.

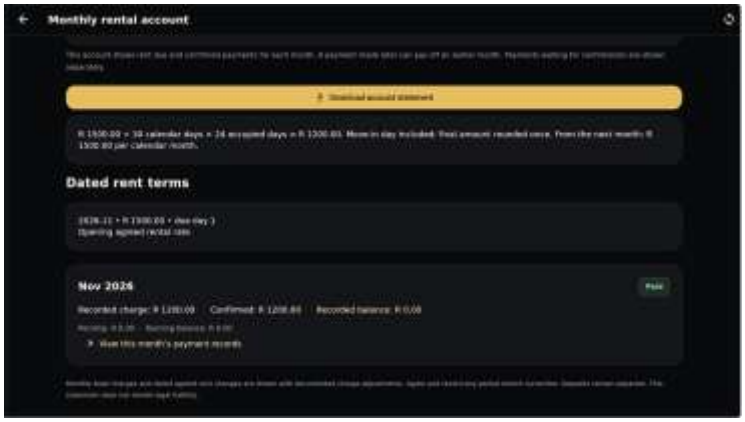



Figure 10. Monthly rent charges and payments.

**2.** For a receipt, open Payments or Payment history, select the confirmed payment and open its receipt. If the balance looks wrong, check the month and payment status before contacting the landlord.

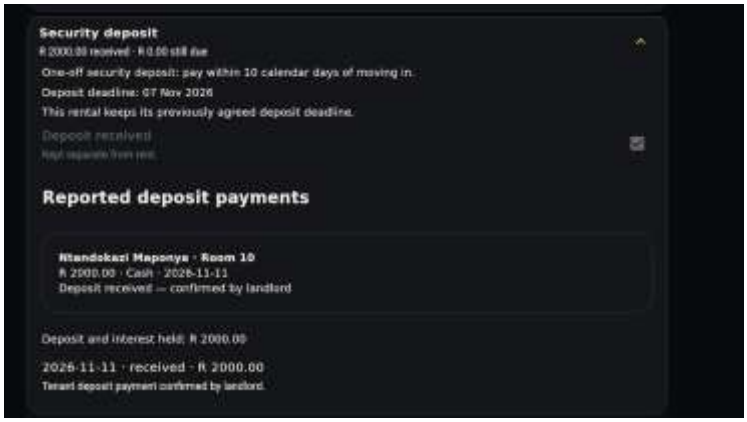



Figure 11. Separate record for the security deposit.

**3.** Open Security deposit to check the deposit amount, deadline and confirmed payments. The app uses a ten-day deposit deadline. Deposit payments are kept separate from monthly rent.

**4.** To report a problem, select Report a problem. Choose where it is, describe what is wrong and send it once. Open the same report to read the landlord's reply or add more information.

**5.** Open Notifications for updates. Acknowledged means the landlord has seen the problem; In progress means it is being worked on; Resolved means it has been marked complete. Reply on the report if you still need help.

Browser alerts are optional. You can read your notifications inside the app even if you do not allow browser alerts.

# Appendix A.1 Structure and roles

Selected source is taken from version 1.5.0, commit cd00a9c. The complete source code and SQL are available in the repository.

## Organised folder structure

lib/main.dart — application and Supabase startup

lib/app.dart — session handling, theme and routing

lib/screens/auth/, landlord/, tenant/, shared/ — role screens and navigation

lib/screens/properties/, payments/, rentals/ — property, money and rental workflows

lib/models/ — typed users, properties, tenancies and payments

lib/data/ — shared state, cloud gateway and accounting calculations

lib/services/ and lib/ui/ — receipts, maps, alerts and visual components

supabase/migrations/ and supabase/tests/ — database rules and database checks

test/ and tool/ — Dart checks, browser checks, builds and release packaging

Source: lib/screens/shared/dashboard.dart, lines 13–24.

```
13  // Select the page from the signed-in account's role.
14  class DashboardScreen extends StatelessWidget {
15    const DashboardScreen({super.key});
16    @override
17    Widget build(BuildContext context) {
18      final user = StoreScope.of(context).currentUser;
19      if (user == null) return const Scaffold(body: SizedBox.shrink());
20      return user.role == Role.landlord
21          ? const LandlordDashboardScreen()
22          : const TenantDashboardScreen();
23    }
24  }
```

Purpose: choose the dashboard from the authenticated account’s role. StoreScope supplies shared state. This screen selection controls presentation; database permissions must separately prevent unauthorised access.

# Appendix A.2 Calendar-day rent

Source: lib/data/research_ledger.dart, lines 69–74.

```
69  /// Calendar-day billing, including the move-in day. Round the final cents only.
70  int daysInMonth(DateTime date) => DateTime(date.year, date.month + 1, 0).day;
71  int proratedFirstCharge(int monthlyCents, DateTime start) {
72    final days = daysInMonth(start), occupied = days - start.day + 1;
73    return (monthlyCents * occupied * 2 + days) ~/ (2 * days);
74  }
```

Purpose: calculate a first partial month using actual month length. The occupied-day count includes move-in day. Amounts use integer cents, and rounding happens once on the final charge rather than on a daily amount.

## Worked example

Monthly rent R1,500; November has 30 days; move-in on 7 November gives 24 occupied days. First rent = R1,500 × 24 ÷ 30 = R1,200. Normal recurring rent remains R1,500 due on the 1st from the following month. A different move-in day produces a different charge.

## Why this matters

Using actual days avoids treating every month as thirty days. Integer cents avoid accumulating floating-point currency errors. This calculation does not record a payment: it creates the charge against which confirmed rent is compared. A property can instead use its selected full-first-month billing rule.

## Rent changes and recorded history

A later agreed rate begins in its effective future month. The previous rate remains in dated history so earlier monthly accounts are not rewritten. Room size and agreement terms can justify different rates; the application does not assume every room must have the same amount or every change applies to an entire year.

# Appendix A.3 Proposed rent split

Source: lib/data/rent_payment_plan.dart, lines 5–27.

```
 5  List<PaymentAllocation> proposeRentPayment({
 6    required int amountCents,
 7    required String firstPeriod,
 8    required Iterable<String> allowedMonths,
 9    required int Function(String) amountStillToReport,
10  }) {
11    var remaining = amountCents;
12    final months =
13        allowedMonths.where((m) => m.compareTo(firstPeriod) >= 0).toSet().toList()
14          ..sort();
15    final result = <PaymentAllocation>[];
16    for (final month in months) {
17      if (remaining <= 0) break;
18      final available = amountStillToReport(month).clamp(0, 2147483647);
19      final part = remaining < available ? remaining : available;
20      if (part > 0) {
21        result.add(PaymentAllocation(month, part));
22        remaining -= part;
23      }
24    }
25    return result;
26  }
```

Purpose: propose how one rent payment is distributed, starting at the tenant's selected month. Allowed months are filtered, sorted and de-duplicated. Each month receives at most the remaining amount needed before money moves to a later month.

For 200,000 cents against 120,000 cents due now, the proposal uses 120,000 cents now and 80,000 cents in the next month. The equivalent rand figures are R2,000, R1,200 and R800.

The function returns a proposal; it does not confirm receipt or mix in a security deposit. The tenant checks the split before submission. The server validates the saved proposal and protects it during landlord review. If the available month range cannot absorb the payment, validation must require a complete valid allocation before saving.

# Appendix A.4 Availability and validation

Source: lib/data/room_availability.dart, lines 5–26.

```
 5  List<int> availableRoomsFor(
 6    RentalProperty property,
 7    Iterable<Tenancy> tenancies,
 8    DateTime moveIn, {
 9    bool actualHandover = true,
10  }) {
11    final day = DateTime(moveIn.year, moveIn.month, moveIn.day);
12    final blocked = <int>{};
13    for (final t in tenancies.where((t) => t.propertyId == property.id)) {
14      final end = actualHandover ? t.endedOn : t.leaseEnd;
15      if (end == null || !DateTime(end.year, end.month, end.day).isBefore(day)) {
16        final room = int.tryParse(t.room);
17        if (room != null) blocked.add(room);
18      }
19    }
20    return [
21      for (var room = 1; room <= property.rooms; room++)
22        if (!blocked.contains(room)) room,
23    ];
24  }
```

Purpose: exclude rooms whose previous rental has not actually ended before the proposed move-in day. The default uses recorded room handover, rather than assuming an agreed lease-end date means the room is empty. Availability is checked again when saving.

## Validation and evidence

The version 1.5.0 release records 105 passing Flutter tests and analysis of 78 Dart files with no issues. Database checks cover migrations and access rules. Chromium workflow checks cover payment allocation, deposits, issue status, notification targets and session behaviour.

# References

Flutter (n.d.) Flutter documentation. Available at: https://docs.flutter.dev/ (Accessed: 13 November 2026).

Netlify (n.d.) Deploy overview. Available at: https://docs.netlify.com/deploy/overview/ (Accessed: 13 November 2026).

Nkambule, M. (2026) BackyardPay source repository, version 1.5.0, commit cd00a9c. Available at: https://github.com/nkambuleinnocent52-Ghost/Mbongiseni-Nkambule-BackyardPay (Accessed: 13 November 2026).

Republic of South Africa (1999) Rental Housing Act 50 of 1999. Available at: https://www.gov.za/documents/rental-housing-act (Accessed: 13 November 2026).

Supabase (n.d.) Auth and Row Level Security documentation. Available at: https://supabase.com/docs/guides/auth and https://supabase.com/docs/guides/database/postgres/row-level-security (Accessed: 13 November 2026).

Walter Sisulu University (2026) Application Technology IV project guideline 2026. Supplied module document.